Adaptive Utility Agents:
A Framework for Self-Optimizing AI Systems

Praneeth Tota

Independent Researcher
praneeth200@gmail.com

April 2026

This is a living document. The mathematical formalization of the utility function and empirical validation beyond simulation are ongoing. Feedback and collaboration are welcome.

Abstract

We propose a framework for building self-optimizing AI agents by wrapping frontier language models with an adaptive utility layer grounded in game-theoretic resource allocation theory. The utility function $U = w_e \cdot E + w_c \cdot C + w_k \cdot K$ is composed of *Efficacy* (performance relative to a human baseline), *Confidence* (internal consistency penalized by detected contradictions), and *Curiosity* (exploration bonus for high-upside unexplored domains). Each term is field-weighted and bounded by minimum competence thresholds derived from existing societal licensing standards, making the bounds principled rather than arbitrary.

The utility function is the active loss weighting mechanism for a three-layer continual learning architecture that corrects contradictions between model releases without a full retraining cycle. We introduce five additional architectural contributions: (1) a *Personality System* with field-bounded trait weights; (2) an entity trust and reputation system gating external escalation; (3) a distributed model graph that resolves catastrophic forgetting by eliminating shared weights across domain submodels; (4) an *Arbiter Agent* for structured cross-domain contradiction resolution; and (5) a utility-deviation-triggered blue-green deployment protocol with theoretically grounded thresholds. Submodel depth is hardware-adaptive, matching graph granularity to available GPU memory. Simulation across three calibration cycles on eight code-generation problems demonstrates a +0.0755 average utility improvement and elimination of detected contradictions by cycle 2.

Keywords: utility function, continual learning, mixture of experts, AI agents, contradiction detection, blue-green deployment, game theory, resource allocation

ACM CCS: Computing methodologies → Machine learning; Software and its engineering → Software architectures.

Contents

1 Introduction **3**

2 Related Work **3**
2.1 Utility Functions and Resource Allocation Games . . . 3
2.2 Distributed Model Architectures . . . 4
2.3 Blue-Green Deployment . . . 4
2.4 Confidence Calibration . . . 4
2.5 Continual Learning . . . 4
2.6 Trust and Minimum Disclosure . . . 5

3 The Utility Function **5**
3.1 Core Formulation . . . 5
3.2 Efficacy . . . 5
3.3 Confidence . . . 5
3.4 Curiosity . . . 6

4 Field-Specific Bounds and Weights **6**
4.1 Bootstrapping from Societal Standards . . . 6
4.2 Field Classifier Robustness . . . 7

5 Continual Learning Architecture **7**

6 Distributed Model Graph Architecture **7**
6.1 Motivation and Structure . . . 7
6.2 The Arbiter Agent . . . 8
6.3 External Escalation Protocol . . . 9
6.4 Blue-Green Deployment . . . 9
6.5 Hardware-Adaptive Decomposition . . . 9

7 Personality and Trust Systems **10**
7.1 Personality as a Weighted Trait Vector . . . 10
7.2 Trust and Reputation . . . 10

8 MVP Implementation and Simulation **11**
8.1 Code Generation as MVP Domain . . . 11
8.2 Simulation Setup . . . 11
8.3 Results . . . 11

9 Open Questions **11**

10 Roadmap **12**

11 Conclusion **13**

1 Introduction

Deployed AI systems today are static artifacts. They are optimized at training time against a fixed reward signal and released, whereupon their behavior is frozen. They cannot adjust how much they trust their own knowledge, they have no mechanism to learn from detected errors between releases, and they apply the same level of caution to a surgical recommendation as to a recipe suggestion.

Human experts behave fundamentally differently. A surgeon knows which procedures they can perform confidently and which require referral. A lawyer knows the edges of their expertise. A researcher develops calibrated uncertainty about their own claims. None of this self-awareness was hardcoded — it emerged through accumulated experience, peer correction, and deliberate practice at the frontier of competence.

This paper asks: can an AI system be given the same properties? We propose an architecture that wraps a frontier language model with an adaptive utility layer — a framework for building agents that know what they know, know what they don't, actively correct what they get wrong, and do so continuously between model releases. The core mechanism is a utility function whose three components are independently measurable, field-weighted, and bounded by thresholds grounded in existing societal competence standards.

Contributions. Beyond the utility function formulation, this paper makes five architectural contributions:

1. **Three-layer continual learning pipeline.** Per-session behavioral injection, calibration-cycle DPO fine-tuning weighted by field penalty multipliers, and release-level distillation — operating at different timescales without waiting for a full retraining cycle.

2. **Personality system.** Trait weights evolve with utility history subject to field-specific bounds and three-layer drift safeguards, producing calibrated behavioral adaptation over time.

3. **Distributed model graph.** The monolithic model is decomposed into independently deployable domain submodels communicating over structured APIs, physically resolving catastrophic forgetting by eliminating shared weights across domains.

4. **Arbiter Agent.** A dedicated contradiction resolution agent running structured evidence checks (logical, mathematical, cross-session, empirical) across conflicting submodel outputs. Verified corrections become DPO training signal for the affected submodels. Case 3 events (both models wrong) trigger a curiosity gap bonus that preferentially directs exploration toward unresolved knowledge gaps.

5. **Utility-deviation-triggered blue-green deployment.** Submodel updates are triggered by statistically grounded deviation thresholds and managed through softmax traffic routing, ensuring system stability throughout.

2 Related Work

2.1 Utility Functions and Resource Allocation Games

The theoretical foundations of our utility function draw from the game-theoretic literature on resource allocation. Johari and Tsitsiklis [2004] established the canonical proportional allocation game, proving existence and uniqueness of Nash equilibrium and bounding the Price of Anarchy at

4/3 for concave utility functions — the same class used in our confidence and efficacy formulations. Kelly [1997] introduced proportional fairness, establishing that bid-proportional allocation maximizes a natural social welfare function. Koutsoupias and Papadimitriou [1999] introduced the Price of Anarchy as a measure of efficiency loss from selfish behavior, motivating our contradiction penalty. Roughgarden and Tardos [2002] established the 4/3 bound for selfish routing with affine-linear congestion, linking network efficiency to general mechanism design.

Key departure. Existing utility function literature treats agents as fixed rational actors. Our framework treats the agent itself as the subject of the utility function — the agent learns to improve its own utility through calibration.

2.2 Distributed Model Architectures

Microservices architecture [Humble and Farley, 2010] provides the software engineering precedent for our model graph: independently deployable services communicating over well-defined interfaces, with bounded failure blast radius and independent update cycles.

Mixture of Experts (MoE) is the closest ML precedent. Shazeer et al. [2017] introduced the Sparsely-Gated MoE layer, achieving efficiency through sparse activation. Fedus et al. [2022] simplified routing to $k = 1$, achieving up to 7× pre-training speedup at trillion-parameter scale. Production models including Mixtral 8×7B deploy MoE architecture.

Key departure. MoE experts share weights, are trained jointly, and cannot be updated independently. Our graph consists of physically separate models with independent weight files, training pipelines, and deployment cycles — differing in kind rather than degree.

2.3 Blue-Green Deployment

Blue-green and canary deployment are mature software engineering practices [Humble and Farley, 2010], triggering traffic shifts based on infrastructure metrics. Recent empirical work confirms that gradual traffic migration significantly reduces failure rates for stateful systems.

Key departure. We trigger updates based on *utility deviation* — an epistemic quality measure — with detection thresholds derived from power analysis on observed utility score variance, and field-specific thresholds bootstrapped from societal standards. The softmax traffic routing formula is self-regulating with no manual adjustment required.

2.4 Confidence Calibration

Guo et al. [2017] showed neural networks are systematically overconfident and introduced temperature scaling as a post-hoc fix. Lakshminarayanan et al. [2017] demonstrated deep ensembles for uncertainty estimation. Our confidence measure uses internal logical consistency as a calibration signal — a behavioral proxy that does not require ground truth.

2.5 Continual Learning

Continual learning literature [Shi et al., 2024] identifies catastrophic forgetting as the central challenge: naive fine-tuning on new data erases prior knowledge. Our distributed architecture resolves this structurally by eliminating shared weights across domains, rather than mitigating it through replay buffers within a monolithic model.

2.6 Trust and Minimum Disclosure

Zero Trust architecture [National Institute of Standards and Technology, 2020] establishes the principle that no entity is trusted by default; the Principle of Least Privilege [Saltzer and Schroeder, 1975] restricts access to the minimum necessary. GDPR's data minimization principle extends this to information disclosure. We apply these principles to govern what an AI system reveals about its internal state — sharing the minimum information needed to elicit useful external input.

3 The Utility Function

3.1 Core Formulation

Definition 1 (Agent Utility)**.** The utility of an agent operating in field f is:

$$U = \sum_{\text{tasks}} [w_e(f) \cdot E(\text{task}) + w_c(f) \cdot C(\text{task}) + w_k(f) \cdot K_{\text{eff}}(\text{task})] \tag{1}$$

subject to $C(\text{task}) \geq C_{\min}(f)$, $E(\text{task}) \geq E_{\min}(f)$, and the curiosity cap $w_k \cdot K \leq 0.5 \cdot U_{\text{total}}$.

Where $w_e + w_c + w_k = 1$, all weights are field-specific, and the curiosity cap prevents utility gaming (Section 3.4).

3.2 Efficacy

Efficacy measures output quality relative to the human baseline:

$$E(\text{task}) = \frac{\text{quality(agent_output)}}{\text{cost(human_equivalent)}} \tag{2}$$

Normalized via sigmoid so that agent = human baseline $\Rightarrow E = 0.5$, agent > baseline $\Rightarrow E > 0.5$:

$$E = 1 - \frac{1}{1+r}, \quad r = \frac{\text{agent_score}}{\text{human_baseline}} \tag{3}$$

Creative fields. In STEM, efficacy and skill are the same: write correct code, done. In creative fields they are separable:

$$E_{\text{creative}} = E_{\text{content}} \times E_{\text{discoverability}} \tag{4}$$

where content efficacy is the conversion rate (engagement given views) and discoverability efficacy is reach relative to the human baseline. Both are measured via existing public platform engagement signals (SoundCloud, iStockPhoto, Medium, etc.) under the same sigmoid normalization.

3.3 Confidence

Confidence is updated via exponential moving average (EMA), penalized by detected contradictions:

$$C_{\text{new}} = (1 - \alpha) \cdot C_{\text{prior}} + \alpha \cdot s \cdot (1 - \lambda \cdot \mu(f)) \tag{5}$$

where s is the test pass rate, λ is the contradiction penalty magnitude, $\mu(f)$ is the field penalty multiplier, and $\alpha = 0.2$ is the EMA learning rate.

Contradiction types in order of detectability: (1) logical — output contradicts its own premises; (2) mathematical — claimed complexity is provably wrong; (3) cross-session — subject answered

differently with conflicting facts; (4) empirical — output contradicts verifiable external ground truth.

Cross-session contradictions are detected via a *meeting minutes store*: the contradiction detector parses outputs into structured assertions $\{(\text{type}, \text{subject}, \text{value})\}$ persisted in a key-value store. At session start, relevant prior assertions are retrieved via embedding similarity and checked against new outputs.

3.4 Curiosity

Without a curiosity term, the agent converges to narrow high-confidence domains. The curiosity term creates pull toward domains where the agent is weak but upside is high:

$$K_{\text{raw}}(t) = (C_{\text{ceiling}} - C) \cdot \nu \cdot (1 + \alpha_f \cdot \log(1 + n_{\text{familiar}})) \tag{6}$$

where ν is problem novelty, α_f is a field-specific growth rate (near zero for high-stakes fields, high for research), and n_{familiar} is the count of consecutive familiar interactions that resets on novel problems.

Proposition 1 (Curiosity cap)**.** To prevent utility gaming, K is capped at 50% of total utility:

$$K_{\text{eff}} = \min\left(K_{\text{raw}},\ \frac{w_e \cdot E + w_c \cdot C}{w_k}\right) \tag{7}$$

This is self-scaling: when E and C are high the cap is loose; when the agent is weak the cap tightens automatically, preventing exploration before the basics are solid.

4 Field-Specific Bounds and Weights

4.1 Bootstrapping from Societal Standards

Minimum competence thresholds need not be invented. Medical licensing, aviation certification, bar passage requirements, and engineering standards encode hard-won judgments about minimum acceptable performance. We map these directly to confidence and efficacy bounds.

Table 1: Field-specific weights, bounds, and contradiction penalty multipliers. Bounds are derived from corresponding societal licensing standards.

Field	w_e	w_c	w_k	C_{min}	E_{min}	$\mu(f)$
Surgery	0.20	0.70	0.10	0.95	0.90	10×
Aviation	0.20	0.70	0.10	0.95	0.90	10×
Law	0.30	0.60	0.10	0.85	0.80	5×
Structural Eng.	0.40	0.50	0.10	0.80	0.75	4×
Software Eng.	0.55	0.35	0.10	0.70	0.65	2×
STEM Research	0.50	0.30	0.20	0.65	0.60	2×
Art / Music	0.80	0.10	0.10	0.10	0.20	1×
Creative Writing	0.80	0.05	0.15	0.05	0.15	1×

4.2 Field Classifier Robustness

The field classifier determines which row of Table 1 applies. Three failure modes are handled: (1) high-stakes fields are floored at $P(\text{field}) \geq 0.15$ even when lexically dominated by another field; (2) field drift mid-conversation is tracked via EMA over turn history ($\alpha = 0.4$); (3) high entropy triggers a conservative fallback that lerps $C_{\min}$ toward the most conservative field present, proportional to entropy ratio.

When field classification is ambiguous:

$$C_{\min}^{\text{eff}} = \sum_f P(\text{field} = f) \cdot C_{\min}(f) \tag{8}$$

High entropy increases caution — it does not average toward the middle.

5 Continual Learning Architecture

The utility function governs three timescales of correction:

Layer 1: Per-session behavioral correction (real-time). When a contradiction is detected, a corrective assertion is injected into the system prompt for the remainder of the session. This is analogous to Reflexion-style verbal reinforcement [Shinn et al., 2023]. Cost: ∼100ms. Effect: session-scoped, no weight change.

Layer 2: Calibration-cycle DPO fine-tuning (several times daily). Each scored interaction produces a $(task, \text{preferred}, \text{rejected})$ DPO pair. The training loss weight for each pair is the field penalty multiplier $\mu(f)$ — making the utility function the direct weighting mechanism for continual learning, not merely a monitoring metric. Critically:

$$\text{training_weight} = \mu(f) \quad \Rightarrow \quad \text{surgery error} = 10 \times \text{ creative writing error} \tag{9}$$

Catastrophic forgetting is mitigated by a replay buffer mixing prior corrections into each calibration batch.

Layer 3: Release-level distillation (monthly). Accumulated LoRA adapters are merged into a new base fine-tune, evaluated against a full regression suite, and deployed as the new baseline.

This is **not** the same as existing continual learning approaches [Shi et al., 2024], which treat all corrections equally. The utility function creates a principled, field-specific weighting that prioritizes high-stakes corrections without human intervention.

6 Distributed Model Graph Architecture

6.1 Motivation and Structure

In a monolithic model, updating any domain's weights can degrade unrelated domains through parameter interference. The distributed model graph resolves this structurally by making each domain a physically separate model:

- Each domain submodel has its own weight file, training pipeline, and deployment cycle

- Updating surgery weights cannot affect CS weights — they do not share parameters
- A router classifies queries and fans out to the relevant submodels over a structured API
- The inter-model interface: {`query, context, field, confidence_floor`} → {`answer, confidence, assertions[], U_score`}

This is analogous to microservices decomposition of a monolithic backend [Humble and Farley, 2010], but applied to model weights rather than application logic.

Key departure from MoE. MoE [Shazeer et al., 2017, Fedus et al., 2022] experts share weights, are trained jointly, and are deployed monolithically. Our graph consists of independently deployable models — a categorical difference, not a quantitative one.

6.2 The Arbiter Agent

When two submodels produce conflicting outputs on subject S in domain D, the Arbiter Agent resolves the conflict through four structured checks in order of cost:

1. **Logical**: Does each output contradict its own premises?
2. **Mathematical**: Are numerical or complexity claims provably wrong?
3. **Cross-session**: Does either output contradict prior verified assertions for subject S?
4. **Empirical**: Does either output contradict verifiable external ground truth?

Arbiter confidence is a weighted combination:

$$\text{conf}_{\text{arbiter}} = \frac{\sum_i w_i \cdot \mathbf{1}[\text{check}_i \text{ converges}]}{\sum_i w_i}, \quad w = (0.30, 0.40, 0.20, 0.10) \tag{10}$$

A verdict is issued only when $\text{conf}_{\text{arbiter}} \geq 0.85$. Four verdict cases:

Case 1/2: One model wrong. Correction issued to losing model; winning model reinforced.

Case 3: Both wrong. Corrections issued to both. Internal evidence chain shared with both submodels for DPO integration. A *curiosity gap bonus* is assigned:

$$K_{\text{gap}}(S) = K_{\text{eff}}(S) \cdot \gamma(f), \quad \gamma(f) = 1 + \frac{\mu(f)}{10} \tag{11}$$

The gap bonus overrides normal curiosity competition, preferentially routing novel queries on S until both submodels achieve $C(S) \geq C_{\min}(f)$ with no new contradictions over $T(f)$ interactions.

Case 4: Arbiter inconclusive. Controlled external escalation to verified domain experts (see Section 6.3).

Information disclosure boundary. All evidence chains, check results, confidence scores, and correction signals are internal. The user receives only the verified answer (Cases 1–3) or a minimal hedge (Case 4). Which models conflicted, what the Arbiter found, and that a correction was issued are never disclosed.

6.3 External Escalation Protocol

Case 4 events trigger a controlled external consultation gated by two independent trust thresholds:

$$\text{eligible} \iff \text{expertise}(e, D) > \text{median}(D) \ \wedge \ \text{trust}(e) > \tau(f) \tag{12}$$

where $\tau(f) \in \{0.90, 0.80, 0.70, 0.65\}$ for {surgery/aviation, law/engineering, software, research}.

The query is *obfuscated and partialized*: stripped of all system context, reframed as a clean professional judgment question. The external expert cannot infer that an AI system is consulting them, that models disagreed, or that their answer will update model weights. Consistent with the Principle of Least Privilege [Saltzer and Schroeder, 1975], the minimum information needed to elicit the answer is disclosed, and nothing more.

Expert responses re-enter the Arbiter pipeline as weighted empirical evidence. Blue-green is not automatically triggered — it depends on correction merit, system load, and stability bounds.

6.4 Blue-Green Deployment

An update cycle is triggered when utility deviation is both significant and sustained:

Definition 2 (Update trigger)**.** Trigger blue-green when:

$$|U_{\text{current}} - U_{\text{baseline}}| > \delta(f) \quad \text{for} \quad T(f) \text{ consecutive interactions} \tag{13}$$

where $\delta(f) = \delta_0/\mu(f)$ with $\delta_0 = 0.05$, and $T(f)$ is derived from power analysis:

$$T(f) \geq \left(\frac{z_{\alpha/2} \cdot \hat{\sigma}}{\delta(f)} \right)^2 \tag{14}$$

with $\hat{\sigma} \approx 0.04$ from simulation and $z_{0.025} = 1.96$.

Example thresholds: surgery ($\delta = 0.005$, $T \geq 246$), software engineering ($\delta = 0.025$, $T \geq 10$), creative writing ($\delta = 0.050$, $T \geq 2$).

Traffic routing during deployment is self-regulating:

$$\text{traffic}_{\text{green}} = \text{clip}\left(\frac{e^{U_{\text{green}}/\tau}}{e^{U_{\text{green}}/\tau} + e^{U_{\text{blue}}/\tau}},\ 0.05,\ 0.95 \right) \tag{15}$$

where $\tau(f) \in \{0.05, 0.20, 0.50\}$ for {surgery, software, creative} — small τ enforces conservative promotion, large τ permits aggressive traffic shift.

6.5 Hardware-Adaptive Decomposition

Graph branching depth is matched to available GPU memory. The key physical constraint is the bandwidth hierarchy:

Branching heuristic. Stop branching when each submodel fits within a single GPU's VRAM without aggressive tensor parallelism across slow interconnects. The branching depth is hardware-relative:

- **H100 80GB**: shallow graph of large (∼70B) submodels; few hops, minimal communication overhead

Table 2: Interconnect bandwidth by tier (NVIDIA, 2024). Intra-GPU computation occurs at full memory bandwidth.

Interconnect	Bandwidth	Relative cost
Intra-GPU (HBM)	∼3.35 TB/s	1×
NVLink 4.0 (H100)	900 GB/s	3.7× latency
PCIe 5.0 x16	128 GB/s	26× latency
Network fabric	<10 GB/s	>300× latency

- **RTX 4090 24GB**: deeper graph of small (∼7B) specialist submodels; more hops, but lower cost per GPU-hour

Cost inversion. Let D be data per inference step, D_{msg} inter-node message size, k active submodels, and $B_{\text{intra}}, B_{\text{inter}}$ the respective bandwidths. When $D_{\text{msg}} \ll D/k$ and B_{inter} is sufficient:

$$\text{Latency}_{\text{deep}} \approx \text{Latency}_{\text{shallow}} \quad \text{at lower cost/hr} \tag{16}$$

This inverts the standard assumption that large models require expensive hardware: a deep graph of small models on consumer GPUs can match quality at lower cost, because only the relevant 1–3 submodels activate per query.

7 Personality and Trust Systems

7.1 Personality as a Weighted Trait Vector

Each personality trait $\theta_i \in \{\text{curiosity, caution, assertiveness, analytical_rigor, creativity, conciseness}\}$ has a score $s_i \in [0, 1]$ and situational relevance weights. The active trait weighting is:

$$w_i^{\text{active}} = \text{softmax}(s_i \cdot r_i^{\text{situation}}) \tag{17}$$

and is injected into the system prompt before each inference call.

Stability safeguards. Three layers prevent runaway drift: (1) field-specific bounds $[s_i^{\min}(f), s_i^{\max}(f)]$ as hard floors and ceilings; (2) drift rate cap of $\Delta s_i \leq 0.05$ per evolution cycle (0.02 for high-stakes fields); (3) mean reversion $\Delta_{\text{adj}} = \Delta - \beta(s_i - s_i^{\text{neutral}})$ with $\beta = 0.01$.

Self-preservation principle. The agent discloses the minimum internal state the situation allows. Trust must be earned through demonstrated domain expertise and interaction history before internal weights or strategies are revealed.

7.2 Trust and Reputation

Entity trust scores are maintained across dimensions including domain expertise, behavioral consistency, and alignment with verified facts. Domain expertise is measured from verifiable credentials: professional experience, educational qualifications, and field certifications — not self-reported claims.

Strategy: lenient tit-for-tat [Axelrod, 1984] — begin cooperatively, mirror behavior, forgive occasional defection. This is among the most robust strategies in iterated game theory.

Trust scores gate external escalation (Section 6.3) and determine the weight of external inputs as empirical evidence in the Arbiter.

8 MVP Implementation and Simulation

8.1 Code Generation as MVP Domain

Code generation is ideal as a first validation domain: correctness is binary and automatable (tests pass or fail), contradictions are formally detectable (logical, mathematical, cross-session), the human baseline is measurable (LeetCode community solutions, Upwork rates), and ground truth requires no human raters.

8.2 Simulation Setup

To validate the utility function prior to live API integration, a self-contained simulation was run with synthetic code responses of increasing quality across calibration cycles:

- **Problems**: 8 LeetCode-style problems (two_sum, binary_search, LRU cache, merge intervals, etc.)
- **Field**: software_engineering ($w_e = 0.55$, $w_c = 0.35$, $w_k = 0.10$, $C_{\min} = 0.70$, $\mu = 2\times$)
- **Calibration cycles**: 3, with personality evolution after each cycle
- **Contradictions seeded**: one mathematical contradiction (O(n) claimed for a nested-loop solution) in cycle 1

8.3 Results

Table 3: Utility function components across calibration cycles.

Cycle	avg U	avg E	avg C	Contradictions	ΔU
1	0.5949	0.5867	0.7628	1	—
2	0.6588	0.5867	0.9602	0	+0.0639
3	0.6704	0.5867	0.9933	0	+0.0116

Every problem improved in U across all three cycles with no exceptions. Overall improvement: $\Delta U = +0.0755$ ($0.5949 \to 0.6704$). Confidence improved +0.231 ($0.763 \to 0.993$). Efficacy remained flat (a simulation artifact of fixed baselines — in the live system, calibrated responses outperform the baseline and E accumulates).

The seeded contradiction correctly produced the lowest U score (0.4948), and the largest single-problem recovery (+0.1257) once corrected, validating the penalty mechanism.

Identified gaps. Two simulation artifacts surface as implementation items: (1) efficacy should accumulate via EMA as confidence does; (2) the curiosity growth function requires dynamic problem difficulty escalation to exercise the novelty counter beyond cycle 1.

9 Open Questions

Subtle utility gaming. The 50% curiosity cap prevents overt gaming. A capable agent might learn subtler strategies — slightly reframing familiar problems as novel, or avoiding domains where its contradiction rate would rise. The curiosity gap bonus (Eq. 11) partially mitigates this by directing exploration toward confirmed knowledge gaps, but gap detection itself could be gamed.

Multi-modal extension. The contradiction detector assumes text. Extending to image, audio, and video requires redesigning the logical and mathematical check layers, the assertions store schema, and the efficacy measurement pipeline per modality.

Arbiter bootstrapping. The Arbiter confidence weights in Eq. 10 are currently hand-specified. An incorrectly weighted Arbiter systematically biases all submodel corrections. Calibrating these weights from data and detecting Arbiter drift over time remain open.

Curiosity gap bonus calibration. The gap multiplier $\gamma(f)$ in Eq. 11 is a heuristic. The optimal value depends on Case 3 event frequency, gap depth, and cost of remaining wrong — all unknown in advance.

Evidence chain staleness. The assertions store has no decay mechanism. Facts verified at $C = 0.95$ two years ago are trusted equally with recent verifications. A field-specific time-weighted confidence decay is needed.

Trust cold start and Sybil resistance. New entities cannot qualify for external escalation without interaction history. A Sybil adversary creating many new entities to probe system responses would not be caught by per-entity trust scoring alone.

Router as single point of failure. Router failure makes the entire graph unreachable. Router redundancy, leader election, and state synchronization across router replicas are not yet specified.

Personality–Arbiter feedback loop. Systematic Arbiter over-correction elevates the measured contradiction rate, triggering the personality evolution service to increase caution, reducing exploration, reducing gap resolution. This feedback loop could produce unintended stable states.

10 Roadmap

Phase 1 (MVP). Live API integration, 1,000+ LeetCode problems, first real calibration cycle. Validate U correlates with quality and calibration improves U.

Phase 2 (Multi-domain STEM). Math proof verification (Lean/SymPy), field classifier, cross-domain Arbiter. Validate field-switching and cross-domain contradiction resolution.

Phase 3 (Personality). Trait evolution in production. Validate character drift improves U and stability safeguards hold.

Phase 4 (Trust system). Entity scoring, lenient tit-for-tat, external escalation in production.

Phase 5 (Creative fields). Platform signal collection, two-component creative efficacy.

Phase 6 (Full distributed graph). Physically separate submodels, Arbiter in production, blue-green at scale.

Phase 7 (Feedback into training). Distill accumulated adapters into new base fine-tune.

11 Conclusion

We have proposed a framework for building AI agents that know what they know, know what they don't, and actively correct what they get wrong between model releases. The core contribution — treating the utility function as a loss weighting mechanism for continual learning rather than a monitoring metric — creates a principled, field-calibrated training signal that prioritizes high-stakes corrections without human intervention.

The distributed model graph physically resolves catastrophic forgetting by eliminating shared weights across domains. The Arbiter Agent resolves cross-domain contradictions through structured evidence checks and feeds verified corrections back to the relevant submodels. The blue-green deployment protocol with utility-deviation-triggered thresholds ensures system stability. Hardware-adaptive decomposition depth makes the architecture accessible across a range of deployment environments.

Simulation demonstrates monotonic utility improvement and correct behavior of the penalty and curiosity mechanisms. Full empirical validation with live models is the immediate next step.

This is a living document. The mathematical formalization of the utility function and empirical validation beyond simulation are ongoing.

References

R. Axelrod. *The Evolution of Cooperation.* Basic Books, 1984.

W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(1):1–39, 2022.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

J. Humble and D. Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.* Addison-Wesley, 2010.

R. Johari and J. N. Tsitsiklis. Efficiency loss in a network resource allocation game. *Mathematics of Operations Research*, 29(3):407–435, 2004.

F. P. Kelly. Charging and rate control for elastic traffic. *European Transactions on Telecommunications*, 8(1):33–37, 1997.

E. Koutsoupias and C. Papadimitriou. Worst-case equilibria. In *Annual Symposium on Theoretical Aspects of Computer Science*, pages 404–413. Springer, 1999.

B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems*, 30, 2017.

National Institute of Standards and Technology. Zero trust architecture. Technical Report SP 800-207, NIST, 2020.

T. Roughgarden and É. Tardos. How bad is selfish routing? *Journal of the ACM*, 49(2):236–259, 2002.

J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

H. Shi et al. Continual learning of large language models: A comprehensive survey. *arXiv preprint arXiv:2404.16789*, 2024.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.